

Traffic Signal Violation Detection System

Complete Project Working, Deep Learning Model Architecture & Video Processing Pipeline Guide

Task: Video Analytics	Model: YOLOv8 Nano (Inference)	Tracking: Spatial IoU Tracker	Speed: Perspective Physics	Violations: Line Intersection
------------------------------	---------------------------------------	--------------------------------------	-----------------------------------	--------------------------------------

1. The Core Question: Are We Training a Model or What?

DIRECT ANSWER: You are NOT training a model in this project. You are performing Real-Time Inference using a Pre-Trained Deep Learning Model (yolov8n.pt). There is no model training, loss computation, or backpropagation script executed here.
--

What is yolov8n.pt? It is YOLOv8 Nano, the most efficient lightweight neural network from Ultralytics. It comes with frozen weights that were pre-trained on the comprehensive **MS COCO (Common Objects in Context)** dataset containing over 330,000 images and 1.5 million object instances. It already possesses generalized visual representations to accurately identify vehicles in diverse road scenes without needing any custom training.

Vehicle Classes Filtered: COCO contains 80 general categories. The detection module (detector.py) filters for the 5 transport-related class IDs: **Car** (ID 2), **Motorcycle** (ID 3), **Bus** (ID 5), **Truck** (ID 7), and **Bicycle** (ID 1). All non-vehicular objects (pedestrians, animals, signs) are automatically ignored.

Comparison: Model Training vs. Project Inference

Aspect	Model Training (Offline / Research)	This Project (Inference Pipeline)
Primary Goal	Learn synaptic weights by adjusting mathematical matrices via gradient descent.	Apply frozen weights to analyze incoming frames and extract bounding boxes in real time.
Hardware Needs	High-end GPUs with massive VRAM (e.g. NVIDIA A100/V100); runs for hours/days.	Runs smoothly on standard consumer CPUs or modest GPUs at 30+ frames per second.
Dataset Needed	Requires tens of thousands of manually annotated images with XML/TXT labels.	Zero custom dataset needed. Uses COCO pre-trained weights directly.
Code Execution	model.train(data='custom.yaml', epochs=100)	results = model.predict(source=frame, conf=0.35) in detector.py
Output Produced	A serialized weights file (best.pt).	Live bounding boxes, speed estimates, violation logs, and annotated video.

2. High-Level Working & Core Objectives

The project acts as an **Automated AI Traffic Surveillance Officer**. In real-world cities, human police officers cannot monitor every intersection 24/7, and traditional inductive loops cut into road asphalt are expensive and prone to physical wear. This software connects to any ordinary IP camera or recorded video file and simultaneously carries out 5 automated tasks:

- **Vehicle Counting & Classification:** Continuously tracks every passing vehicle and tallies statistics for cars, trucks, buses, and two-wheelers.
- **Speed Estimation:** Computes physical speed in km/h for every moving vehicle using frame displacement and perspective-aware camera calibration.
- **Traffic Signal State Tracking:** Synchronizes with current traffic signal phases (Green, Yellow, Red) automatically via timer, camera ROI, or manual remote.
- **Red-Light Violation Detection:** Mathematically detects if a vehicle breaches the designated stop line while the traffic light is RED.
- **Automated Evidence Snapshot (E-Challan):** Crops high-resolution pictures of offending vehicles, stamps them with legal audit metadata (time, vehicle ID, speed, violation), and stores them for law enforcement.

3. How the Video is Processed (In Simple Words)

To understand how video is processed, remember that a video is simply a fast sequence of static photos called **frames** (usually 30 photos per second). The system in pipeline.py processes each frame through 7 sequential stages:

Step 1: Reading Frame & Signal Status

The video feed is ingested frame-by-frame via OpenCV (cv2.VideoCapture). For each frame, signal_detector.py determines the current state of the traffic light. In **AUTO_CYCLE** mode, it computes the exact elapsed video second using the frame index (frame_idx / fps) and checks the light cycle (e.g., 12s Green -> 3s Yellow -> 12s Red). In **COLOR_ROI** mode, it crops a designated traffic light box in the image and runs an HSV color filter to spot whether Red, Yellow, or Green pixels dominate.

Step 2: Deep Learning Vehicle Detection (YOLOv8)

The raw image is passed into the YOLOv8 neural network (detector.py). In a single forward pass, the model scans the frame and predicts bounding box coordinates [x1, y1, x2, y2], confidence percentages, and class labels for all vehicles. Detections below the confidence threshold (0.35) are filtered out to eliminate background noise.

Step 3: Vehicle Tracking & Identity Assignment

Object detection alone is 'stateless'—it sees boxes in Frame 1 and boxes in Frame 2, but does not know if they belong to the same car. The tracker (tracker.py) solves this by computing **Intersection over Union (IoU)** between previous boxes and new boxes. When a vehicle is first spotted, it gets a unique ID (e.g. Vehicle #12). As long as it moves across the screen, it keeps that same ID, and its road contact point (the bottom-center anchor of the bounding box) is saved into its trajectory history.

Step 4: Speed Calculation (Physics & Perspective Correction)

In speed_estimator.py, the system looks at where the vehicle's road contact point was 10 frames ago versus where it is now.

Using basic physics: **Speed = Distance / Time**.

- **Time:** Calculated as elapsed_frames / FPS (e.g., 10 frames at 30 FPS = 0.33 seconds).
- **Distance:** Calculated in pixels and converted to real meters using a calibration scale (meters_per_pixel). A *perspective factor* is applied: vehicles far away (near top of screen) represent more physical meters per pixel than vehicles right in front of the camera (bottom of screen). The final velocity is output in **km/h**.

Step 5: Violation Rule Checking (Stop Line Crossing & Speeding)

In violation_detector.py, two geometric rules are tested on every active vehicle:

- **Signal Jump:** The stop line is defined as a line segment AB. The vehicle's motion from the previous frame to current frame forms segment CD. A 2D orientation cross-product test checks if segment CD intersected segment AB. If an intersection occurred while the traffic light was **RED**, a *Signal Jump Violation* is immediately flagged.
- **Speeding Violation:** If the calculated speed exceeds the designated limit (e.g. > 50 km/h), an *Overspeeding Violation* is flagged.

Step 6: Automated Evidence Capture (E-Challan Snapshot)

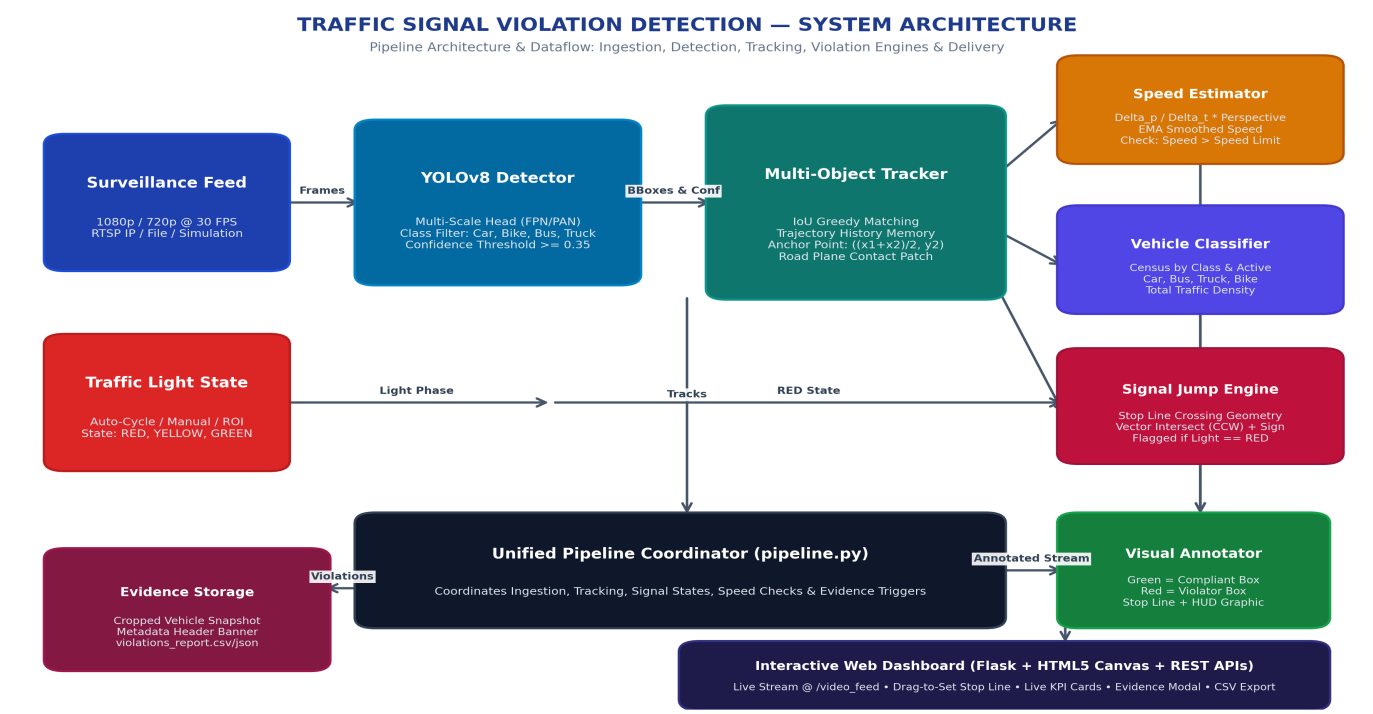
When a violation is flagged for the first time, the system automatically crops the bounding box of the violating car from the original frame. It adds an 80-pixel informational header banner with high-contrast text stating: **Violation Type, Vehicle ID, Speed, Date, and Timestamp**. The picture is saved into the violations/ folder as legal evidence, and the incident is appended to the audit log.

Step 7: Rendering HUD & Video Stream

In annotator.py, the frame is enriched with visual overlays: green/red vehicle bounding boxes, speed tags, trajectory tails, the colored stop line (glows green during green signal, red during red signal), and a glassmorphism telemetry dashboard at the top. The finished frame is encoded into a high-speed JPEG stream and served to the web browser dashboard.

4. Architecture & Pipeline Flowchart

Figure 1: End-to-End System Processing Pipeline



Summary of Pipeline Stages & Data Transformations

Stage	Input Data	Processing Logic	Output Produced
1. Ingestion	MP4 Video / Camera Feed	OpenCV frame demuxing & FPS synchronization	Raw BGR image frame (NumPy array)
2. Signal Check	Frame + Video Timestamp	Time modulo cycle or HSV color segmentation	State enum: RED, YELLOW, GREEN
3. Detection	Raw BGR image frame	YOLOv8 CNN forward pass with COCO filter	List of bounding boxes & classes
4. Tracking	Current & historical boxes	IoU spatial matching & track persistence	List of TrackedVehicle with unique IDs
5. Velocity	Vehicle anchor trajectory	Pixel displacement / Δt with perspective scale	Calibrated speed in km/h
6. Enforcement	Trajectory + Stop line + Signal	2D Line segment cross-product intersection	Violation alerts & cropped snapshot files
7. Output	Frame + Statistics + Alerts	Visual annotation & MJPEG HTTP streaming	Live dashboard UI & downloadable video

5. Codebase Module Map: Where Everything Lives

Module File	Primary Class / Function	Responsibility in the Project
detector.py	VehicleDetector	Loads yolov8n.pt and predicts vehicle bounding boxes for COCO classes.
tracker.py	VehicleTracker TrackedVehicle	Maintains persistent vehicle IDs across frames using IoU tracking and records trajectory points.
speed_estimator.py	SpeedEstimator	Converts pixel movement over consecutive frames into real-world km/h with perspective compensation.

signal_detector.py	TrafficSignalManager	Manages signal state via automatic timer cycles, manual dashboard override, or HSV color detection.
violation_detector.py	ViolationDetector	Computes 2D vector line crossings against the stop line on RED and triggers evidence image cropping.
annotator.py	VisualAnnotator	Draws bounding boxes, dynamic stop line, speed indicators, and the HUD banner onto frames.
pipeline.py	TrafficViolationPipeline	Master orchestrator linking detector, tracker, speed estimator, signal, and violation modules.
app.py	Flask Server	Web dashboard providing real-time video stream, interactive stop line placement, and violation gallery.

6. Project Defense & Viva Quick Reference (FAQ)

Use these concise, authoritative explanations during project demonstrations, technical interviews, or viva exams:

Q1: If someone asks 'Did you train the model?', what should you say?

Answer: 'No, we utilized a pre-trained state-of-the-art YOLOv8 Nano model trained on the COCO dataset for object detection inference. Vehicle detection is an already solved generalized problem, so our engineering contribution lies in the real-time multi-stage pipeline: spatial-temporal vehicle tracking, perspective-compensated speed estimation, 2D vector geometry for stop-line breach detection, and automated evidence generation.'

Q2: How is vehicle speed calculated from an ordinary 2D CCTV camera without radar?

Answer: 'We use the fundamental relationship $Speed = Distance / Time$. Time is derived from video frame rate (FPS). Distance is derived from tracking the bottom-center anchor point of the vehicle on the road plane across a 10-frame sliding window. Because perspective causes objects near the camera to move more pixels per meter than distant objects, we apply a vertical perspective scaling factor ($perspective_factor = 1.0 / (0.4 + 0.6 * y_ratio)$) combined with a camera calibration scale (meters per pixel) to compute accurate km/h.'

Q3: How does the system detect whether a car crossed the line specifically during a RED light?

Answer: 'We treat the stop line as a 2D line segment AB and the vehicle anchor's motion between consecutive frames as line segment CD. We apply a Counter-Clockwise (CCW) orientation test using cross products to determine if segments AB and CD intersect. If an intersection is detected, the system immediately queries the TrafficSignalManager. If the signal state is RED at that exact instant, a Signal Jump violation is recorded.'

Q4: Why use YOLOv8 instead of older architectures like Haar Cascades or Faster R-CNN?

Answer: 'Haar cascades rely on handcrafted edge features that fail under lighting variations and occlusions. Faster R-CNN uses a two-stage Region Proposal Network (RPN) which is accurate but computationally heavy (often under 10 FPS on CPU). YOLOv8 is a single-stage detector utilizing an anchor-free split head and C2f feature aggregation, delivering high Mean Average Precision (mAP) while operating in real-time at 30+ frames per second.'

Q5: Can this system be fine-tuned or trained on custom datasets if needed in the future?

Answer: 'Yes! If the system needs to recognize specialized local vehicles (e.g., auto-rickshaws, e-rickshaws, or emergency ambulances), we can take the same YOLOv8 architecture and perform Transfer Learning by fine-tuning on a labeled Indian/local traffic dataset with `model.train(data='custom_traffic.yaml', epochs=50)`.'